UNIVERSITAT DE BARCELONA
Facultat de Matemàtiques i Informàtica

GRAU DE MATEMÀTIQUES

Treball final de grau

GRAPH NEURAL NETWORKS FOR AUTONOMOUS THEOREM PROVING

Autor: Carlos Fernández Lorán

Director: Dr. Carles Casacuberta
Realitzat a: Departament of Mathematics and Computer Science

Barcelona, April 27, 2026

Contents

Abstract **ii**

1 Introduction **1**

2 Simple Type Theory **3**

3 Generalized Algebraic Theories **5**
3.1 Formal Definition 7
3.2 Models and Homomorphisms 9

4 Calculus of Constructions **11**
4.1 Syntactic Formulation 11
4.2 Doctrines of Constructions 15

5 Expansion Algorithm **19**
5.1 Problem Definition 19
5.2 Expansion algorithm 20

Bibliography **27**

Abstract

The main goal of this work is to prove the existence of.....

Notation: We will work over an algebraically closed field K of characteristic zero.

2020 Mathematics Subject Classification. 11G05, 11G10, 14G10

Chapter 1

Introduction

One of the central challenges in mathematics and computer science is the automatic generation of proofs. This problem has driven the development of proof assistants as Lean, Rocq (formerly Coq) and Agda, systems that allow mathematical theorems to be stated and verified mechanically. In the recent years, due to the increase in capabilities of artificial intelligence models, specially the surge of large language models, there's been a surge in the interest for this problem.

A natural formal setting in which to study this problem is type theory, a foundational framework for mathematics that assigns types to mathematical objects. Type theory was originally introduced to resolve the paradoxes in naive set theory, but it has since evolved into a rich and self-standing foundation for constructive mathematics.

Chapter 2

Simple Type Theory

Chapter 3

Generalized Algebraic Theories

In this chapter we will present the notion of Generalized Algebraic Theory (or GAT), originally presented by J. Cartmell in his thesis [1] [2], abstracted from versions of Martin-Löf type theory and is a generalization of the usual notion of a many-sorted algebraic theory in the following manner: while in Martin Löff's many-sorted algebraic theories sorts are constant types, interpreted as sets, in generalized algebraic theories sorts might be constant or variable, over a specified range of types, and variable types are interpreted as families of sets. So a generalized algebraic theory consists of:

1. a set of sorts, that can be constant or variable;

2. a set of operators, with the sort of the arguments and value specified;

3. a set of axioms, where each axiom is an identity between similar expressions.

Let's present category theory for example: The set of sorts contains two elements Ob and Hom. Ob is a constant type and Hom is a variable type, for a pair of terms of type Ob, name them x, y, we have a type $Hom(x, y)$. Here we see a key point in general algebraic theories, types vary over terms, no over types. The set of operators also has two elements id and $\circ$. The id operator has one argument x of type Ob, and returns a value of type $Hom(x, x)$. This shows how the type of value of an operator might vary depending on the arguments. The $\circ$ operator (or composition) expects 5 arguments, three of type Ob, name them x, y, z, one of type $Hom(x, y)$, say f, and one of type $Hom(y, z)$, named g, and it returns a value of type $Hom(x, z)$. Here we see that the type of an argument might also vary depending on a previous argument. Informally, instead of writing $\circ(x, y, z, f, g)$ we can write $\circ(f, g)$, as the first three arguments might be inferred from the last two. This notation is discussed in the original paper [2].

Now we will introduce the notation we will be using when talking about terms and types. This notation will be formally defined in a later section. So consider you have a set of variables V, we will associate terms to variables as required. When asserting that a term t is of type Δ we will write $t : \Delta$, but if t has variables $x_1, \ldots, x_n$ occurring in it this assertion does not make sense unless we assign some types to $x_1, \ldots, x_m$. To do this we will write:

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$$

that reads as: if x_1 is a variable of type Δ_1, ...and if x_n is a variable of type Δ_n, then t is a term of type Δ. Similarly, if we want to assert that if x_1 is a variable of type Δ_1, ...and if x_n is a variable of type Δ_n, then Δ is a type, we can write:

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$$

this assertions are called rules. Rules are used to express which expressions are well-formed. We will work with this rules as units instead of the base expressions. This is because terms and types do not make sense by themselves but they need to be thought of in a context.

Axioms are also written as rules. The conclusion of those rules are equalities between types or between terms of the same type. We write those rules as:

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta = \Delta'} \qquad \frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t = t' : \Delta}$$

and read as: if x_1 is a variable of type Δ_1, ...and x_n is a variable of type Δ_n, then $\Delta = \Delta'$, or $t = t'$ as terms of type Δ, respectively.

To continue with the previous example, we can write the axioms of category theory as:

$$\begin{aligned}
&x, y : Ob, f : Hom(x, y), \quad \circ(id(x), f) = f \\
&x, y : Ob, f : Hom(x, y), \quad \circ(f, id(y)) = f \\
&w, x, y, z : Ob, f : Hom(w, x), g : Hom(x, y), h : Hom(y, z), \\
&\qquad \circ(\circ(f, g), h) = \circ(f, \circ(g, h))
\end{aligned}$$

When presenting a theory's language i.e. the sets of sorts and operators, we must present each symbol with a rule specifying the role of this symbol, either as a sort symbol or as specifically typed operator symbol. This rules are called introductory rules. In the case of a sort symbol A the rule should be like

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{A(x_1, \ldots, x_n) \text{ is a type}}$$

clearly specifying the types A is dependent on. In the case of an operator symbol, a rule of the form

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{f(x_1, \ldots, x_n) : \Delta}$$

suffices to specify the types of the arguments and the value of f.

Finally, to present a theory we will give a set os symbols, each associated to its introductory rule, and a set of axioms. And, of course, everything must be well-formed, something we will address in the following section. So now we can give a formal presentation of the theory of categories:

Symbol	*Introductory Rule*
Ob	Ob is a type.
Hom	$x, y \in \mathrm{Ob} : \mathrm{Hom}(x, y)$ is a type.
$\circ$	$x, y, z \in \mathrm{Ob}$, $f \in \mathrm{Hom}(x, y)$, $g \in \mathrm{Hom}(y, z) : \circ(f, g) \in \mathrm{Hom}(x, z)$.
id	$x \in \mathrm{Ob} : \mathrm{id}(x) \in \mathrm{Hom}(x, x)$.

Axioms

$\circ(\mathrm{id}(x), f) = f$,	whenever $x, y \in \mathrm{Ob}$ and $f \in \mathrm{Hom}(x, y)$.
$\circ(f, \mathrm{id}(y)) = f$,	whenever $x, y \in \mathrm{Ob}$ and $f \in \mathrm{Hom}(x, y)$.
$\circ(\circ(f, g), h) = \circ(f, \circ(g, h))$,	whenever $w, x, y, z \in \mathrm{Ob}$, $f \in \mathrm{Hom}(w, x)$, $g \in \mathrm{Hom}(x, y)$ and $h \in \mathrm{Hom}(y, z)$.

3.1 Formal Definition

In this section we will formally define all of the concepts we just introduced. To give a notion of well-formed introductory rule, we need the concept of drivability, which also depends on the introductory rules. To tackle this problem we will leave aside the question of well-formedness until we have the complete set of derived rules of a theory. We will first define a pretheory and then define a theory as a well-formed pretheory.

We will assume to have countable set of variables V. Lets define the notion of rule on an alphabet W, the elements of W are called symbols.

Definition 3.1. *The set of* ***expressions*** *is defined inductively such that expressions are finite sequences of elements of $W \cup V \cup \{(\} \cup \{)\} \cup \{,\}$ defined by the clauses:*

1. *if $x \in V$, then x is an expression;*
2. *if $L \in W$, then L is an expression;*
3. *if $L \in W$ and $e_1, \ldots, e_n$ are expressions, then $L(e_1, \ldots, e_n)$ is an expression.*

Now we can define the building blocks of a rule:

Definition 3.2. *A **premise** is defined to be a sequence of $V\times$ the set of expressions. The empty sequence is allowed and we call it an empty premise. If $((x_1, \Delta_1), \ldots, (x_n, \Delta_n))$ is a premise, we write it as $x_1 : \Delta_1, \ldots, x_n : \Delta_n$.*

Definition 3.3. *We call a **conclusion** to one of the following:*

1. *T-conclusion: defined by a single expression Δ and written as 'Δ is a type'.*
2. *$\in$-conclusion: defined by a pair of expressions (t, Δ) and written as '$t \in \Delta$'.*
3. *T =conclusion: defined by a pair of expressions (Δ_1, Δ_2) and written as '$\Delta_1 = \Delta_2$'.*
4. *$\in$=conclusion: defined by a triple of expressions (t_1, t_2, Δ) and written as '$t_1 = t_2 : \Delta$'*

Finally a **rule** is determined by a premise P and a conclusion C, and is written as $\frac{P}{C}$. We name the rule as T-rule, $\in$-rule, T =rule or $\in$=rule according to the form of the conclusion.

Now we can define pretheory and later theory.

Definition 3.4. *A **pretheory** consists of*

1. *a set of sort symbols S;*
2. *a set of operator symbols Z;*
3. *for each sort symbol A, an associated rule of the alphabet $S \cup Z$:*

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{A(x_1, \ldots, A_n) \text{ is a type}}$$

   for some $n \geq 0$. It is called the introductory rule of A;

4. *for each operator symbol F, an associated rule of the alphabet $S \cup Z$:*

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{F(x_1, \ldots, x_n) : \Delta}$$

   for some $n \geq 0$. It is called the introductory rule of F;

5. *a set of axioms. Each axiom is either a T =rule or a $\in$=rule of the alphabet $S \cup Z$.*

Now we will tackle the problem of well-formedness of rules.

Definition 3.5. *If U is a pretheory, then*

1. *a* ***context*** *is a premise* $x_1 : \Delta_1, \ldots, x_n : \Delta_n$ *such that the rule*

$$\frac{x_1 : \Delta_1, \ldots, x_{n-1} : \Delta_{n-1}}{\Delta_n \text{ is a type}}$$

is a derived rule such that x_n *is distinct from all of* $x_1, \ldots, x_n$.

2. (a) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ *is* ***well-formed*** *if the premise* $x_1 : \Delta_1, \ldots, x_n : \Delta_n$ *is a context.*

   (b) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$ *is* ***well-formed*** *if* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ *is a derived rule of* U.

   (c) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta = \Delta'}$ *is* ***well-formed*** *if* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$ *and* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta' \text{ is a type}}$ *are derived rules of* U.

   (d) *the rule* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t = t' : \Delta}$ *is* ***well-formed*** *if* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$ *and* $\dfrac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t' : \Delta}$ *are derived rules of* U.

Definition 3.6. *The set of* ***derived rules*** *of a pretheory* U *is the set of rules derivable by the following principles of derivation:*

Definition 3.7. *A pretheory is well-formed if all of its introductory rules are well-formed. A* ***theory*** *is a well-formed pretheory.*

Lastly, we will introduce substitution. Given $\Delta, t_1, \ldots, t_n$ expressions and $x_1, \ldots, x_n$ variables, then we define the expression $\Delta[t_1|x_1, \ldots, t_n|x_n]$ the result of simultaneous replacement of $x_1, \ldots, x_n$ in Δ by $t_1, \ldots, t_n$. We call this the Substitution operation. Then, as proven in the original paper [2], the set of derived rules of a theory is closed under the operation of substitution of correctly typed terms for variables. We could add substitution as one of the derivation rules but would make applying induction very messy.

3.2 Models and Homomorphisms

With all the syntax defined, we will consider the semantics of a theory. To do this, we will introduce the notions of model and model homomorphism.

First, we will set some notation and basic concepts.

Definition 3.8. *A tree is a pair* $(N, father)$ *where* N *is a not empty set of nodes and* $father : N \to N$ *such that:*

1. *for all $A \in N$ there exists $n \in \omega$ such that $father^n(A) = father^{n+1}(A)$;*

2. *for all $A, B \in N$, if $A = father(A)$ and $B = father(B)$, then $A = B$.*

We will write $A \trianglelefteq B$ if $A = father(B)$ and $A \lhd B$ if $A \trianglelefteq B$ and $A \neq B$. By condition 2, there is a unique least element of N, call it 1. Finally we will define, for any $A \in N$, $level(A)$ to be the least $n \in \omega$ with $father^n(A) = father^{n+1}(A)$.

The set of contexts of a theory is structured as a tree. Whit the empty context as it's least element and $father(\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle) = \langle x_1 : \Delta_1, \ldots, \Delta_{n-1} \rangle$.

We will also want to consider the tree of sets, families of sets, families of families of sets, ... Consider the following notation for families: given a set A, if for every element $a \in A$ $B(A)$ is a set, then the corresponding A-indexed family of sets is denoted as $\lambda a \in A.B(a)$ or as $\lambda a.B(a)$; If we have a set $C(a, b)$ for every $a \in A$ and $b \in B(a)$, then $\lambda a \in A.\lambda b \in B(a).C(a, b)$ is an A-indexed family of families of sets; and so on. The collection of sets, families of sets, families of families of sets, and so on, is called the **tree of families**. Given $1 \lhd A_1 \lhd A_2 \lhd \cdots \lhd A_n \lhd A$ in the tree of families. Then, for all $a_1 \in A_1$, ..., for all $a_n \in A_n(a_1, \ldots, a_{n-1})$, $A(a_1, \ldots, a_n)$ is a set. Finally we should define the concept of **operator**. If we have $1 \lhd A_1 \lhd \cdots \lhd A_n \lhd A$ in the tree of families. If for all $a_1 \in A_1$, ..., for all $a_n \in A_n(a_1, \ldots, a_{n-1})$, $f(a_1, \ldots, a_n) \in A(a_1, \ldots, a_n)$. We say that $\lambda a_1 \in A_1 \ldots \lambda a_n \in A_n(a_1, \ldots, a_{n-1}).f(a_1, \ldots, a_n)$ is an operator at A.

Chapter 4

Calculus of Constructions

4.1 Syntactic Formulation

In this section, we will formally define the syntax of Calculus of Constructions. We will use the approach of Thomas Streicher in [4] as the original definition in [3] is characterized by its syntax overloading, making it much harder to follow. They just use $[x : A]B$ to present product types, functional abstraction and universal quantification. Also, properties are presented as terms of type $Prop$ and as types themselves. To solve the latter, we will be using a type constructor $Proof$ that given a proposition $p : Prop$ it will give $Proof(p)$ the type of proofs of proposition p.

First, we will define the class of **pre-well-formed type expressions**, ranged over by capital letters, and the class of **pre-well-formed object expressions**, ranged over by lower case letters, by mutual recursion in a BMF-like way:

$$
\begin{array}{rll}
E ::= & (\Pi x : E)\, E & \text{product of types} \\
 & \mid \ \mathrm{Prop} & \text{type of propositions} \\
 & \mid \ \mathrm{Proof}(t) & \text{type of proofs of proposition } t
\end{array}
$$

$$
\begin{array}{rll}
t ::= & x & \text{variable} \\
 & \mid \ (\lambda x\colon E)t & \text{functional abstraction} \\
 & \mid \ \mathrm{App}([x\colon E]E, t, t) & \text{function application} \\
 & \mid \ (\forall x\colon E)t & \text{universal quantification}
\end{array}
$$

We say a **pre-context** is a syntactical expression of the form $x_1 : A_1, \ldots, x_n : A_n$, where $x_1, \ldots, x_n$ are pairwise syntactically different variables. Contexts are ranged over by capital greek letters. We define $\mathrm{Var}(\Gamma)$ as the set of variables declared in Γ. We will not distinguish expressions which are equivalent up to renaming of

variables (α-conversion), to do this we will be using *deBruijn indices* [1]. We define a **pre-judgment** to be a syntactic expression of one of the following forms:

$A\text{type}$	$A$is a type
$A = B$	A and B are equal types
$t : A$	t is an object of type A
$t = s : A$	t and s are equal objects of type A

we will use the letter $\mathcal{J}$ to refer to pre-judgments. We define a **pre-sequent** as a syntactic expression of the following form

Γok	Γ is a context
$\Gamma = \Delta$	Γ and Δ are equal contexts
$\Gamma \Rightarrow \mathcal{J}$	the judgement $\mathcal{J}$ holds w.r.t. Γ

Finally we define the set of **sequents**, i.e. valid pre-sequents, to be generated by the following set of rules. Sequents are ranged over by capital italic letters. This set of rules is more extent that the original in order to make explicit rules for handling contexts and to add more rules stating equality of types and objects.

When presenting the axiomatization of Calculus of Constructions we shall use the following notation:

$$\frac{S_1}{\overline{\overline{S_2}}} \text{ abbreviates } \frac{S_1}{S_2} \text{ and } \frac{S_2}{S_1}$$

Context Formation Rules:

$$\textsc{Empty}\quad \frac{}{\text{ok}}$$

$$\textsc{Cont-Intro}\quad \frac{\Gamma \Rightarrow A\text{type}}{\overline{\overline{\Gamma, x\colon A\text{ok}}}}, \text{ if } x \notin \text{Var}(\Gamma)$$

$$\textsc{Cont-Thin}\quad \frac{\Gamma, \Delta\text{ok} \quad \Gamma \Rightarrow A\text{type}}{\Gamma, x\colon A, \Delta\text{ok}}, \text{ if } x \notin \text{Var}(\Gamma) \cup \text{Var}(\Delta)$$

$$\textsc{Cont-Sub}\quad \frac{\Gamma, x\colon A, \Delta\text{ok} \quad \Gamma \Rightarrow t\colon A}{\Gamma, \Delta[x \mid t]\text{ok}}$$

Context Equality Rules

$$\textsc{Crefl}\quad \frac{\Gamma = \Gamma}{\overline{\overline{\Gamma\text{ok}}}}$$

$$\textsc{Csym}\quad \frac{\Gamma = \Delta}{\Delta = \Gamma}$$

[1] *deBruijn indices* are a method to refer to declarations not by name (variables) but by numbers which tell how far one has to go back into the current context of an expression to find the declaration of the thing it refers to.

$$\textsc{Ctrans}\ \frac{\Gamma = \Delta \quad \Delta = \Theta}{\Gamma = \Theta}$$

$$\textsc{Cequ}\ \frac{\Gamma, x\colon A, \Delta\mathrm{ok} \quad \Gamma \Rightarrow A = B}{\Gamma, x\colon A, \Delta = \Gamma, x : B, \Delta}$$

$$\textsc{Creflect1}\ \frac{\Gamma, x\colon A = \Delta, x\colon B}{\Gamma = \Delta}$$

$$\textsc{Creflect2}\ \frac{\Gamma, x\colon A = \Delta, x\colon B}{\Gamma \Rightarrow A = B_1}$$

Structural Rules for Judgement Formation

$$\textsc{Ass}\ \frac{\Gamma, x\colon A, \Delta\mathrm{ok}}{\Gamma, x\colon A, \Delta \Rightarrow x\colon A}$$

$$\textsc{Reflection1}\ \frac{\Gamma \Rightarrow \mathcal{J}}{\Gamma\mathrm{ok}}$$

$$\textsc{Reflection2}\ \frac{\Gamma \Rightarrow t : A}{\Gamma \Rightarrow A\mathrm{type}}$$

$$\textsc{Thin}\ \frac{\Gamma, \Delta \Rightarrow \mathcal{J} \quad \Gamma \Rightarrow A\mathrm{type}}{\Gamma, x\colon A, \Delta \Rightarrow \mathcal{J}}, \text{ if } x \notin \mathrm{Var}(\Gamma) \cup \mathrm{Var}(\Delta)$$

$$\textsc{Sub}\ \frac{\Gamma, x\colon A, \Delta \Rightarrow \mathcal{J} \quad \Gamma \Rightarrow t : A}{\Gamma, \Delta[x \mid t] \Rightarrow \mathcal{J}[x \mid t]}$$

Equality Rules:

$$\textsc{Equ}\ \frac{\Gamma = \Delta \quad \Gamma \Rightarrow \mathcal{J}}{\Delta \Rightarrow \mathcal{J}}$$

$$\textsc{Refl-Typ}\ \frac{\Gamma \Rightarrow A = A}{\Gamma \Rightarrow A\mathrm{type}}$$

$$\textsc{Refl-Obj}\ \frac{\Gamma \Rightarrow t = t : A}{\Gamma \Rightarrow t : A}$$

$$\textsc{Symm-Typ}\ \frac{\Gamma \Rightarrow A = B}{\Gamma \Rightarrow B = A}$$

$$\textsc{Symm-Obj}\ \frac{\Gamma \Rightarrow t = s : A}{\Gamma \Rightarrow s = t : A}$$

$$\textsc{Trans-Typ}\ \frac{\Gamma \Rightarrow A = B \quad \Gamma \Rightarrow B = C}{\Gamma \Rightarrow A = C}$$

$$\textsc{Trans-Obj}\ \frac{\Gamma \Rightarrow t_1 = t_2 : A \quad \Gamma \Rightarrow t_2 = t_3 : A}{\Gamma \Rightarrow t_1 = t_3 : A}$$

$$\textsc{Repl1}\ \frac{\Gamma \Rightarrow t = s : A \quad \Gamma, x\colon A, \Delta\mathrm{ok}}{\Gamma, \Delta[x \mid t] = \Gamma, \Delta[x \mid t]}$$

$$\textsc{Repl2}\ \frac{\Gamma \Rightarrow t = s : A \quad \Gamma, x\colon A, \Gamma \Rightarrow B\mathrm{type}}{\Gamma, \Delta[x \mid t] \Rightarrow B[x \mid t] = B[x \mid s]}$$

$$\textsc{Repl3}\ \frac{\Gamma \Rightarrow t_1 = t_2 : A \quad \Gamma, x\colon A, \Delta \Rightarrow s : B}{\Gamma, \Delta[x \mid t_1] \Rightarrow s[x \mid t_1] = s[x \mid t_2] : B[x \mid t_1]}$$

$$\textsc{Conv1}\ \frac{\Gamma \Rightarrow A = B \quad \Gamma \Rightarrow t : A}{\Gamma \Rightarrow t : B}$$

$$\textsc{Conv2}\ \frac{\Gamma \Rightarrow A = B \quad \Gamma \Rightarrow t = s : A}{\Gamma \Rightarrow t = s : B}$$

Rules for Products of types

$$\prod\text{-}\textsc{Form}\ \frac{\Gamma, x\colon A \Rightarrow B\mathrm{type}}{\Gamma \Rightarrow (\prod x\colon A)B\mathrm{type}}$$

$$\prod\text{-}\textsc{Equ}\ \frac{\Gamma \Rightarrow A_1 = A_2 \quad \Gamma, x\colon A_1 \Rightarrow B_1 = B_2}{\Gamma \Rightarrow (\prod x\colon A_1)B_1 = (\prod x\colon A_2)B_2}$$

$$\prod\text{-}\textsc{Intro}\ \frac{\Gamma, x\colon A \Rightarrow t : B}{\Gamma \Rightarrow (\lambda x\colon A)t : (\prod x\colon A)B}$$

$$\xi\text{-}\textsc{Rule}\ \frac{\Gamma, x\colon A_1 \Rightarrow t_1 = t_2 : B \quad \Gamma \Rightarrow A_1 = A_2}{\Gamma \Rightarrow (\lambda x\colon A_1)t_1 = (\lambda x\colon A_2)t_2\colon (\prod x\colon A_1)B}$$

$$\prod\text{-}\textsc{Elim}\ \frac{\Gamma \Rightarrow t\colon (\prod x\colon A)B \quad \Gamma \Rightarrow s\colon A \quad \Gamma, x\colon A \Rightarrow B\mathrm{type}}{\Gamma \Rightarrow \mathrm{App}([x\colon A]B, t, s)\colon B[x \mid s]}$$

$$\prod\text{-}\textsc{Elim-Equ}\ \frac{\Gamma \Rightarrow A_1 = A_2 \quad \Gamma \Rightarrow t_1 = t_2\colon (\prod x\colon A_1)B_1 \quad \Gamma, x\colon A_1 \Rightarrow B_1 = B_2 \quad \Gamma \Rightarrow s_1 = s_2\colon A_1}{\Gamma \Rightarrow \mathrm{App}([x\colon A_1]B_1, t_1, s_1) = \mathrm{App}([x\colon A_2]B_2, t_2, s_2)\colon B_1[x \mid s_1]}$$

$$\beta\text{-}\textsc{Rule}\ \frac{\Gamma, x\colon A \Rightarrow B\mathrm{type} \quad \Gamma, x\colon A \Rightarrow t\colon B \quad \Gamma \Rightarrow s\colon A}{\Gamma \Rightarrow \mathrm{App}([x\colon A]B, (\lambda x : A)t, s) = t[x \mid s]\colon B[x \mid s]}$$

$$\eta\text{-}\textsc{rule}\ \frac{\Gamma, x\colon A \Rightarrow B\mathrm{type} \quad \Gamma \Rightarrow t\colon (\prod x\colon A)B}{\Gamma \Rightarrow (\lambda x\colon A)\mathrm{App}([x\colon A]B, t, x) = t\colon (\prod x\colon A)B}$$

Rules for Propositions

$$\textsc{Prop}\ \frac{\Gamma\mathrm{ok}}{\Gamma \Rightarrow \mathrm{Proptype}}$$

$$\textsc{Prop-Type}\ \frac{\Gamma \Rightarrow p\colon \mathrm{Prop}}{\Gamma \Rightarrow \mathrm{Proof}(p)\mathrm{type}}$$

$$\textsc{Prop-Equ}\ \frac{\Gamma \Rightarrow p_1 = p_2\colon \mathrm{Prop}}{\Gamma \Rightarrow \mathrm{Proof}(p_1) = \mathrm{Proof}(p_2)}$$

$$\forall\text{-Intro}\quad \frac{\Gamma, x\colon A \Rightarrow p\colon \mathrm{Prop}}{\Gamma \Rightarrow (\forall x\colon A)p\colon \mathrm{Prop}}$$

$$\forall\text{-Equ}\quad \frac{\Gamma, x\colon A_1 \Rightarrow p_1 = p_2\colon \mathrm{Prop} \quad \Gamma \Rightarrow A_1 = A_2}{\Gamma \Rightarrow (\forall x\colon A_1)p_1 = (\forall x\colon A_2)p_2\colon \mathrm{Prop}}$$

$$\forall\text{-Elim}\quad \frac{\Gamma, x\colon A \Rightarrow p \in \mathrm{Prop}}{\Gamma \Rightarrow \mathrm{Proof}((\forall x\colon A)p) = (\prod x\colon A)\mathrm{Proof}(p)}$$

We will write $\vdash \mathcal{J}$ to say that $\mathcal{J}$ is a derivable judgement.

4.2 Doctrines of Constructions

We will leverage the concept of context categories introduced in the earlier chapter to serve as a basic structure for handling dependent types, and add more conditions so that it can handle products of families of types and the type of properties, in order to develop a categorical structure that captures the interpretation of the Calculus of constructions.

First, we will introduce product types into our semantics, but to do so, we need the notion of section of an object:

Definition 4.1. *Let $\mathcal{C}$ be a contextual category, A and B be objects of $\mathcal{C}$ such that $A \lhd B$, the set of **sections** of B is defined as*

$$\mathit{Sections}(B) := \{s\colon A \to B \mid \ p(B) \circ s = \mathit{id}_A\}$$

If $\mathrm{level}(B) = 1$, then

$$\mathrm{Sections}(B) = \{s \mid \ s\colon 1 \to B\}$$

as any morphism $s\colon 1 \to B$ satisfies $p(B) \circ s = \mathrm{id}_1$ as 1 is a terminal object. In this case, the notion of section matches the notion of *global element* of B.

Definition 4.2. *A **contextual category with products of families of types** is a category $\mathcal{C}$ together with two mappings $\prod$ and eval defined over the collection of objects B of $\mathcal{C}$ with $\mathit{level}(B) \geq 2$, such that:*

1. *whenever $1 \lhd A \lhd B$, then $1 \lhd \prod(B)$ and $\mathit{eval}_B\colon p(A)^* \prod(B) \to B$ is a morphism over A*

$$\begin{tikzcd}
p(A)^* \prod(B) \arrow[rr, "\mathit{eval}_B"] \arrow[dr, "p(p(A)^* \prod(B))"'] & & B \arrow[dl, "p(B)"] \\
& A &
\end{tikzcd}$$

such that for all $s \in Sections(B)$ there is a unique section $t \in Sections(\prod(B))$ with $eval_B \circ p(A)^*t = s$. This unique section t is called $curry(s)$.

$$\begin{tikzcd}
A \arrow[r] \arrow[d, "p(A)^*t"] \arrow[dd, bend right=60, "id_A"'] & 1 \arrow[d, "t"'] \arrow[dd, bend left=60, "id_1"] \\
p(A)^*\prod(B) \arrow[r, "q(p(A),\prod(B))"'] \arrow[d, "eval_B"] \arrow[u, bend left, "s"] & \prod(B) \arrow[d] \\
B \arrow[d] & \\
A \arrow[r] & 1
\end{tikzcd}$$

Observation 4.3. *as any section t of $\prod(B)$ gives a section $eval_B \circ p(A)^*t$ of B, hence we have a canonical 1-1-correspondence of $Sections(B)$ and $Sections(\prod(B))$.*

2. *and for any object J and any morphism $f : J \to 1$ the following conditions are satisfied*

$$f^*(\prod(B)) = \prod(f^*B) \quad \textit{and} \quad f^*(eval_B) = eval_{f^*B}$$

It follows from the definition that substitution preserves functional abstraction or currying:

Lemma 4.4. *Let $\mathcal{C}$ be a contextual category with products of families of types. Let $1 \lhd A \lhd B$ be objects and $f : J \to 1$ a morphism in $\mathcal{C}$. If t is a section of B, then*

$$f^*(curry(B)) = curry(f^*B)$$

Notice that in any contextual category with products of families of types, the following equations hold:

$$\mathrm{eval}_B \circ p(A)^*\mathrm{curry}(s) = s \tag{β}$$

$$\mathrm{curry}(\mathrm{eval}_B \circ p(A)^*t) = t \tag{η}$$

The first equation corresponds to the β-rule of typed λ-calculi and the second equation corresponds to the η-rule of typed λ-calculi. Contextual Categories with products correspond to λ-calculus with dependent types.

Now we will add the type of propositions. To do it, we shall find objects in our category whose sections corresponds, i.e. its terms, correspond to objects.

Definition 4.5. *Let $\mathcal{C}$ be a context category, an* ***enumeration of types of*** *$\mathcal{C}$ is an object T of $\mathcal{C}$ with level$(T) = 2$.*

An enumeration of types T of $\mathcal{C}$ is called a ***collection of types*** *of $\mathcal{C}$ if, and only if, for all objects A and morphisms $f, g\colon A \to$ father(T)*

$$f^*T = g^*T \quad \Longrightarrow \quad f = g$$

it is the object father(T) that we are interested in.

Definition 4.6. *a* ***doctrine of constructions*** *is a contextual category with products of families of types together with a collection of types T such that for all $A, B \in Ob\mathcal{C}$ with $A \lhd B$ and $f\colon B \to Prop$, where $Prop =$ father$T$, there exists a necessarily unique morphism $g\colon A \to Prop$ also denoted by $\forall(f)$ such that*

$$g^*T = \forall(f)^*T = \prod(f^*T)$$

Now we will see that doctrines of constructions correspond to models of Calculus of Constructions.

Chapter 5

Expansion Algorithm

5.1 Problem Definition

In this section we will introduce a formal definition of our problem, proposed at the beginning of the thesis, and reformulate it to an equivalent problem easier to solve.

Our problem might be formalized in the following manner: given a theory U and a derived rule of U of the form

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{\Delta \text{ is a type}}$$

we want to find an expression of U such that the rule

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n}{t : \Delta}$$

is a derived rule of U.

This problem is undecidable, in general, as it follows from the undecidability of the inhabitation in a GAT. This is why we won't try to find an algorithm that either finds a term or says that the type is empty. We aim to find an algorithm that in the case that the type is inhabited, it will find a term in a finite number of steps. In the case that the type is empty, it does not have to give an output.

We can consider this problem from the point of view of the context category $C(U)$, where it is rephrased as: given a context $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$ in $C(U)$, we want to find a morphism

$$\langle x_1, \ldots, x_n, t \rangle : \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle \to \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n, x : \Delta \rangle$$

in $C(U)$. Then if we name $A = \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle$, this is equivalent to find a morphism $\langle t_1, \ldots, t_n, t \rangle : \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n \rangle \to \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n, x : \Delta \rangle$ in the

category C_A, as by the following commutative diagram

$$\begin{tikzcd} A \arrow[rr, "\langle t_1,\ldots,t_n,t\rangle"] \arrow[dr, "id_A"'] & & \langle x_1 : \Delta_1, \ldots, x_n : \Delta_n, x : \Delta\rangle \arrow[dl, "\langle x_1,\ldots,x_n\rangle"] \\ & A & \end{tikzcd}$$

for each $1 \leq i \leq n$, $x_i = t_i$. This restriction holds for all morphism $f = \langle t_1, \ldots, t_n, t_{n+1}, \ldots, t_{n+m}\rangle : B \to B'$ in C_A, by the commutativity of the diagram

$$\begin{tikzcd} B \arrow[rr, "f"] \arrow[dr] & & B' \arrow[dl] \\ & A & \end{tikzcd}$$

$t_i = x_i$ for all $1 \leq i \leq n$. That's why we will adopt the notation $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m\rangle_A$ to represent $\langle x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Omega_m\rangle$ and $f = \langle t_{n+1}, \ldots, t_{n+m}\rangle_A$ to represent $\langle x_1, \ldots, x_n, t_{n+1}, \ldots, t_{n+m}\rangle$ in C_A.

5.2 Expansion algorithm

In this section we will expose the expansion algorithm, that will be responsible of finding the $\langle t\rangle_A$ in C_A.

First consider G a finite subcategory of C_A such that it contains $\langle\rangle_A$, $\langle x : \Delta\rangle_A$ and $p(\langle x : \Delta\rangle_A) : \langle x : \Delta\rangle_A \to \langle\rangle_A$. The process will consist in the construction of a sequence $\{G_i\}_i$ of finite subcategories of C_A such that $G_0 = G$ and that G_{n+1} is a finite subcategory of C_A computed via the process described below from G_n and where $G_n \subseteq G_{n+1}$ for all $n \geq 0$. We say that an object/morphism of C_A is **constructible by expansion** if there exists a $n \geq 0$ such that the given object/morphism is contained in G_n. The process of constructing G_{n+1} from G_n is consists on applying the following rules to each context $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m\rangle_A$ of G_n.

E1 For each sort symbol A with a well-formed introductory rule

$$\frac{z_1 : \Lambda_1, \ldots, z_n : \Lambda_n}{A(z_1, \ldots, z_n) \text{ is a type}}$$

such that G_n contains morphisms

$$\begin{aligned}
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A &\xrightarrow{\langle y_1,\ldots,y_m,t_1\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1 \rangle_A \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A &\xrightarrow{\langle y_1,\ldots,y_m,t_2\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_2 : \Lambda_2[t_1|z_1] \rangle_A \\
&\vdots \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A &\xrightarrow{\langle y_1,\ldots,y_m,t_w\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_w : \Lambda_w[t_1|z_1, \ldots, t_{w-1}|z_{w-1}] \rangle_A
\end{aligned}$$

then we add the context $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, y : A(t_1, \ldots, t_w) \rangle_A$ and its projection to G_{n+1}.

E2 add the morphisms

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1,\ldots,y_m,y_i\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, y : \Omega_i \rangle_A$$

and their corresponding projections for each $0 \leq i \leq n$.

E3 for every operator symbol F with a well-formed introductory rule

$$\frac{z_1 : \Lambda_1, \ldots, z_n : \Lambda_n}{F(z_1, \ldots, z_n) : \Lambda}$$

if the following morphisms exist

$$\begin{aligned}
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A &\xrightarrow{\langle y_1,\ldots,y_m,t_1\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1 \rangle_A \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A &\xrightarrow{\langle y_1,\ldots,y_m,t_2\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_2 : \Lambda_2[t_1|z_1] \rangle_A \\
&\vdots \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A &\xrightarrow{\langle y_1,\ldots,y_m,t_w\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_w : \Lambda_w[t_1|z_1, \ldots, t_{w-1}|z_{w-1}] \rangle_A
\end{aligned}$$

then we add the morphism

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1,\ldots,y_m,F(t_1,\ldots,t_w)\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_w : \Lambda[t_1|z_1, \ldots, t_w|z_w] \rangle_A$$

E4 for every morphism

$$\langle t_1, \ldots, t_{m-1} \rangle_A : \langle z_1 : \Lambda_1, \ldots, z_w : \Lambda_w \rangle_A \to \langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1} \rangle_A$$

we construct the corresponding pullback

$$\begin{tikzcd}
\langle z_1 : \Lambda_1, \ldots, z_w : \Lambda_w, y_m : \Omega_m[t_1|y_1, \ldots, t_{m-1}|y_{m-1}]\rangle_A \arrow[r, "\langle t_1,\ldots,t_{m-1},y_m\rangle_A"] \arrow[d] & \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m\rangle_A \arrow[d] \\
\langle z_1 : \Lambda_1, \ldots, z_w : \Lambda_w\rangle_A \arrow[r, "\langle t_1,\ldots,t_{m-1}\rangle_A"] & \langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}\rangle_A
\end{tikzcd}$$

in G_{n+1}.

Once we have applied the expansion rules to all of the contexts, we complete G_{n+1} with the composition morphisms to ensure it is a subcategory.

Theorem 5.1. *Every object and every morphism of C_A is constructible by expansion.*

Proof. We will proof this result by structural induction over the underlying derived rules corresponding to such object (context) or morphism (operator).

Consider a context $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m\rangle_A$ of U, which is the representative of an object in $C(U)$. This means, by the definition of context, that its underlying rule

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}}{\Omega_m \text{ is a type}} \tag{5.1}$$

is a derived rule of U and y_m is a variable different to $x_1, \ldots, x_n, y_1, \ldots, y_{m-1}$. Hence, the premise $\langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}\rangle_A$ is a context and by induction it is constructible by expansion, i.e. there exists $n_0 \in \mathbb{N}$ such that $\langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}\rangle_A \in G_{n_0}$. As (5.1) is a derived rule, it must have been derived by the only derivation rule that has a T-rule as a conclusion: CF2(a). Hence, there exists a sort symbol S with a well-formed introductory rule $\dfrac{z_1 : \Lambda_1, \ldots, z_w : \Lambda_w}{S(z_1, \ldots, z_w) \text{ is a type}}$ and derived rules

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}}{t_1 : \Lambda_1},$$
$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}}{t_2 : \Lambda_2[t_1|z_1]},$$
$$\vdots$$
$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}}{t_w : \Lambda_w[t_1|z_1, \ldots, t_{w-1}|z_{w-1}]}$$

By induction hypothesis their corresponding morphisms

$$\langle y_1, \ldots, y_{m-1}, t_j\rangle_A : \langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}\rangle_A \to \\ \langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}, z_j : \Lambda[t_1|z_1, \ldots, t_{j-1}|z_{j-1}]\rangle_A$$

are constructible by expansion, so there exist in G_{n_j} for some $n_j \in \mathbb{N}$.

Take $n = \max\{n_0, n_1, \ldots, n_j\}$. Then, all of the objects, and morphisms mentioned belong to G_n so by the expansion rule E1,

$$\langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1} \rangle_A \longleftarrow \langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}, y_m : S(t_1, \ldots, t_w) \rangle_A$$

is included in G_{n+1}. And by the equivalence of intended equivalence of denotation

$$\langle y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}, y_m : S(t_1, \ldots, t_w) \rangle_A \equiv \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A$$

so $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A$ is constructible by expansion.

Now consider an operator

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle t_1, \ldots, t_w \rangle} \langle z_1 : \Lambda_1, \ldots, z_w : \Lambda_w \rangle_A$$

a representative of a morphism in C_A. By induction hypothesis we know that both $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A$ and $\langle z_1 : \Omega_1, \ldots, z_w : \Omega_w \rangle_A$ are constructible by expansion.

Moreover, as $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A$ is an operator, for each $1 \leq j \leq w$, the rule

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Omega_m}{t_j : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}]} \tag{5.2}$$

is a derived rule of U. Now we will see that the following morphisms are constructible by expansion:

$$\begin{aligned}
&\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_1 \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1 \rangle_A \\
&\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_2 \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_2 : \Lambda_2[t_1|z_1] \rangle_A \\
&\qquad\qquad\qquad\qquad \vdots \\
&\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_w \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_w : \Lambda_w[t_1|z_1, \ldots, t_{w-1}|z_{w-1}] \rangle_A
\end{aligned}$$

To do this, we will use how any of the rules (5.2) are derived. To do it, we must use one of the following derivation rules: T1, CF1 or CF2(b). Hence,

- if derived by T1, we have that both

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Omega_m}{\Delta = \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}]}$$

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Omega_m}{t_j : \Delta}$$

are derived rules of U. So by the first rule, due to the equivalence realization of identity of denotation, we know that

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Delta \rangle_A \equiv \\ \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}] \rangle_A$$

hence they are the same object in C_A. And by the second one, applying induction, we have that

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_j \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Delta \rangle_A$$

is constructible by expansion, so as $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Delta \rangle_A$ and $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}] \rangle_A$ are the same object in C_A, then

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_j \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}] \rangle_A$$

is constructible by expansion.

- if derived by CF1, we have that

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_{m-1} : \Omega_{m-1}}{\Omega_m \text{ is a type}}$$

is a derived rule of U, where y_m is a variable distinct from all $x_1, \ldots, x_n$, $y_1, \ldots, y_{m-1}$, and that there exists an i, $1 \leq i \leq m$, such that

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Omega_m}{t_j : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}]} \equiv \frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Omega_m}{y_i : \Omega_i}$$

Then, by induction hypothesis, the context $\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle$ is constructible by expansion. By applying the expansion rule E2 and an argument of equivalence similar to the one above, we get that

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_j \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}] \rangle_A$$

is constructible by expansion.

- if derived by CF2(b), there exists an operator symbol F with a well-formed introductory rule

$$\frac{c_1 : \Gamma_1, \ldots, c_v : \Gamma_v}{F(c_1, \ldots, c_v) : \Gamma}$$

and derived rules

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Delta_m}{s_1 : \Gamma_1}$$
$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Delta_m}{s_2 : \Gamma_2[s_1|c_1]}$$
$$\vdots$$
$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Delta_m}{s_v : \Gamma_v[s_1|c_1, \ldots, s_{v-1}|c_{v-1}]}$$

such that

$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Delta_m}{\Gamma[s_1|c_1, \ldots, s_v|c_v] = \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}]}$$
$$\frac{x_1 : \Delta_1, \ldots, x_n : \Delta_n, y_1 : \Omega_1, \ldots, y_m : \Delta_m}{t_j = F(s_1, \ldots, s_v) : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}]}$$

are derived rules of U. By the equivalence relation of identity of denotation, we have to prove that the morphism

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, F(s_1, \ldots, s_v) \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Gamma[s_1|c_1, \ldots, s_v|c_v] \rangle_A$$

is constructible by expansion. By induction we have that the following morphisms are constructible by expansion:

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, s_1 \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, c_1 : \Gamma_1 \rangle_A$$
$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, s_2 \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, c_2 : \Gamma_2[s_1|c_1] \rangle_A$$
$$\vdots$$
$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, s_v \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, c_v : \Gamma_v[s_1|c_1, \ldots, s_{v-1}|c_{v-1}] \rangle_A$$

Hence, by applying expansion rule E3, we have that

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, F(s_1, \ldots, s_v) \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Gamma[s_1|c_1, \ldots, s_v|c_v] \rangle_A$$

is constructible by expansion as we wanted.

Now we will show tha the morphism

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_1, \ldots, t_w \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1, \ldots, z_w : \Lambda_w \rangle_A$$

is constructible by expansion. To do this we will prove that given a j, $2 < j \leq n$, if

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1, \ldots, y_m, t_1, \ldots, t_{j-1} \rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1, \ldots, z_{j-1} : \Lambda_{j-1} \rangle_A$$

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1,\ldots,y_m,t_j\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Lambda_j[t_1|z_1, \ldots, t_{j-1}|z_{j-1}]\rangle_A$$

are constructible by expansion so is

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m \rangle_A \xrightarrow{\langle y_1,\ldots,y_m,t_1,\ldots,t_j\rangle_A} \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1, \ldots, z_j : \Lambda_j - 1\rangle_A$$

By expansion rule E4, the following diagram is constructible by expansion:

$$\begin{tikzcd}
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_j : \Lambda_j[t_1|z_1, \ldots, t_j|z_j]\rangle_A \arrow[r, "\langle y_1,\ldots,y_m,t_1,\ldots,t_{j-1},z_j\rangle_A"] \arrow[d] & \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1, \ldots, z_j : \Lambda_j\rangle_A \arrow[d] \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m\rangle_A \arrow[r, "\langle y_1,\ldots,y_m,t_1,\ldots,t_{j-1}\rangle_A"] & \langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1, \ldots, z_{j-1} : \Lambda_{j-1}\rangle_A
\end{tikzcd}$$

and by composition we get that

$$\langle y_1, \ldots, y_m, t_1, \ldots, t_{j-1}, z_j\rangle_A \circ \langle y_1, \ldots, y_m, t_j\rangle_A = \langle y_1, \ldots, y_m, t_1, \ldots, t_j\rangle_A$$

is constructible by expansion. It remains to see that the morphsim

$$\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1, \ldots, z_w : \Lambda_w\rangle_A \xrightarrow{\langle z_1,\ldots,z_w\rangle_A} \langle z_1 : \Lambda_1, \ldots, z_w : \Lambda_w\rangle$$

is constructible by expansion. That comes directly from the fact that $\langle z_1 : \Lambda_1, \ldots, z_w : \Lambda_w\rangle_A$ is a context and the repeated application of the rule E4 as this diagram is constructible by expansion:

$$\begin{tikzcd}
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m z_1 : \Lambda_1, \ldots, z_w : \Omega_w\rangle_A \arrow[r, "\langle z_1,\ldots,z_w\rangle_A"] \arrow[d] & \langle z_1 : \Lambda_1, \ldots, z_w : \Omega_w\rangle_A \arrow[d] \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m z_1 : \Lambda_1, \ldots, z_{w-1} : \Omega_{w-1}\rangle_A \arrow[r, "\langle z_1,\ldots,z_{w-1}\rangle_A"] \arrow[d] & \langle z_1 : \Lambda_1, \ldots, z_{w-1} : \Omega_{w-1}\rangle_A \arrow[d] \\
\vdots \arrow[d] & \vdots \arrow[d] \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m, z_1 : \Lambda_1\rangle_A \arrow[r, "\langle z_1\rangle_A"] \arrow[d] & \langle z_1 : \Lambda_1\rangle_A \arrow[d] \\
\langle y_1 : \Omega_1, \ldots, y_m : \Omega_m\rangle \arrow[r, "\langle\rangle_A"] & \langle\rangle_A
\end{tikzcd}$$

□

Bibliography

[1] John Cartmell, *Generalized Algebraic Theories and Contextual Categories*, Ph.D. thesis, Oxford University, 1978.

[2] _____, *Generalised algebraic theories and contextual categories*, Annals of Pure and Applied Logic **32** (1986), 209–243.

[3] Thierry Coquand and Gérard Huet, *The calculus of constructions*, Information and Computation **76** (1988), no. 2, 95–120.

[4] Thomas Streicher, *Semantics of Type Theory Correctness, Completeness and Independence Results*, 1991.